KD-Tree Based Smart Robot Object Location System

BLAST 02 — Research and Technical Report

Assigned topic: KD-Tree | Emerging area: AI / Robotics

Student: Akashkumar Dilipbhai Patel
Instructor: Dr. Victor Govindaswamy

1. Introduction

For this assignment, I studied a data structure called a KD-Tree and how it can help with location searching. My assigned area is AI and Robotics, so I connected the idea to a simple warehouse robot. A robot may need to know where boxes, shelves, or a charging station are placed. If the locations are stored as points, a KD-Tree can help find a nearby point without always checking every object one by one.

I made a menu-based C++ program called “KD-Tree Based Smart Robot Object Location System.” It is a small learning project, not a real robot. The user adds objects, sets a robot position, and asks the program to find the nearest object or measure distance to a selected object.

2. What Is a KD-Tree?

KD stands for k-dimensional. A KD-Tree is a binary tree used to organize points that have coordinates. The “k” tells us how many values describe each point. In my project, I use two values, X and Y, so it is a 2D KD-Tree. Every node stores an object name and its location. A node can connect to a left child and a right child.

A normal binary search tree usually compares one key. In my 2D KD-Tree, the comparison changes between X and Y as we move down the levels. This is the main idea that makes it useful for location data [1].

3. How the Data Is Organized

My program uses these simple rules: at depth 0, compare X; at depth 1, compare Y; at depth 2, compare X again. This pattern continues. A smaller value on the current comparison axis goes to the left. Otherwise, the point goes to the right. If an object already exists at exactly the same X and Y position, my program does not add a duplicate.

Object	X	Y
Boxes	4	7
Shelf B	2	6
Workstation	1	5
Shelf A	8	3

Example KD-Tree after adding the objects in the order shown above:

```
Boxes (4,7)  [compare X]
├── L: Shelf B (2,6)  [compare Y]
│    └── L: Workstation (1,5)  [compare X]
└── R: Shelf A (8,3)  [compare Y]
```

Figure 1. Simple 2D KD-Tree using my project objects. At the root, Shelf B goes left because 2 is less than 4 on the X-axis. Shelf A goes right because 8 is greater than 4. Under Shelf B, Workstation goes left because its Y value, 5, is less than 6.

4. Main KD-Tree Operations

Insertion

When the user adds an object, the program starts at the root and compares the correct coordinate at each level. It moves left or right until it finds an empty position and adds a new node. For example, Shelf B (2,6) goes left of Boxes (4,7) because the root compares X.

Exact location search

If the user searches for (8,3), the program follows the same X/Y comparison rules to find Shelf A. If the point is not stored, it shows that no object was found. This is different from searching by name: my program identifies an object using its coordinates.

Nearest-object search

This is the most important operation for my robotics example. The robot gives its current location, and the program looks for the stored object with the shortest straight-line distance. It checks a likely branch first. When needed, it also checks the other branch, because a closer point might be across the dividing line. This is called backtracking [1, 2].

Removing an object

My project uses an easy-to-follow removal method. First, it checks that the object exists. Next, it collects the other objects, clears the old tree, and inserts the remaining objects again. This rebuild method is simple for a small student project, but it takes more work than a specialized KD-Tree deletion method.

5. Distance Calculation Example

The program uses Euclidean distance, which means straight-line distance between two points. The formula is:

$$\text{Distance} = \sqrt{[(\text{objectX} - \text{robotX})^2 + (\text{objectY} - \text{robotY})^2]}$$

For example, if the robot is at (6,5) and Shelf B is at (2,6):

$$\text{Distance} = \sqrt{[(2 - 6)^2 + (6 - 5)^2]} = \sqrt{(16 + 1)} = \sqrt{17} \approx 4.12 \text{ units.}$$

If the robot is at (6,5), the nearest object from my four example objects is Boxes (4,7), at about 2.83 units. This is only straight-line distance. My program does not calculate a driving path or avoid obstacles.

6. Time and Space Complexity

A reasonably balanced KD-Tree can avoid searching every point. An uneven tree may be much slower. The estimates below describe the operations in my program [1, 2].

Operation	Typical / balanced case	Worst case
Insert	O(log n)	O(n)
Exact search	O(log n)	O(n)
Nearest neighbor	Often checks fewer than n points; no fixed guarantee	O(n)
Display all objects	O(n)	O(n)
Remove (my rebuild method)	O(n log n) if rebuilding stays balanced	$O(n^2)$ if reinsertion becomes very uneven

Here, n means the number of objects. The tree needs O(n) memory to store n objects. Recursive operations also use extra call-stack memory depending on tree height. My removal method additionally stores a temporary list of the remaining objects, so it needs O(n) extra memory.

7. Advantages and Limitations

Advantages

- It organizes objects using their X and Y locations instead of keeping only a plain list.
- It supports exact location search and nearest-neighbor search.
- It can avoid visiting some parts of the tree when looking for a nearby object.
- The same basic idea can be used with more than two coordinates.

Limitations

- If objects are added in an unlucky order, the tree can become uneven and searches may slow down.
- The nearest-neighbor search may still need to visit many nodes in difficult cases.
- My deletion method rebuilds the tree, so it is not ideal for a very large or constantly changing environment.
- My program uses 2D integer coordinates and does not include real sensors, obstacle detection, or route planning.

8. Real-World Uses of KD-Trees

Use 1: Robotics and spatial sensing

Robotics systems can work with many points that describe nearby space, such as points collected from a depth sensor or a 3D scan. Finding points close to a given position is useful in spatial processing. KD-Trees are one way to organize such points and perform nearest-neighbor searches. My warehouse example uses the same location-search idea, but with simple 2D points instead of real sensor data.

Use 2: Maps and geographic locations

A map application may need to search among many locations, such as service points or landmarks. KD-Trees can help with searching for points that are close to a given coordinate. For this use, the points and distance calculation must match the map coordinate system; my classroom example uses a flat X/Y grid.

Use 3: Machine learning and data analysis

KD-Trees are also useful when a system needs to compare a new point with nearby stored points. For example, nearest-neighbor search can be part of grouping similar data points. The sources explain how KD-Trees organize coordinates and search nearby points [1, 2]. My project focuses on a robot instead of a machine-learning model.

9. Connection to AI and Robotics

The main connection is that a robot needs useful information about nearby objects. In a warehouse, the robot might want to know which box is closest, how far away a charging station is, or whether an object is stored at a given location. My KD-Tree stores the object positions, while the Robot class stores the robot's own position separately.

A real robot would also need sensors, movement control, and safe path planning. My application only demonstrates the location-search part. I chose this smaller example because it lets me clearly show how the KD-Tree works in C++.

10. My C++ Application and Design

I divided the project into three C++ files: KDTree.h for the declarations, KDTree.cpp for the KD-Tree functions, and main.cpp for the menu and robot location. The KDTree class manages the nodes. The Robot class uses setLocation(), getX(), getY(), and hasLocation(). The Object and Node structures hold object information and tree links.

The program has nine menu options: add an object, search by location, set or update the robot location, find the nearest object, measure distance to a selected object, display all objects, display the tree, remove an object, and exit. I also included checks for invalid input, repeated coordinates, missing objects, and trying to search before setting the robot location.

11. Sample Use and Edge Cases

For a simple demonstration, I can add Boxes (4,7), Shelf B (2,6), Workstation (1,5), and Shelf A (8,3). After setting the robot to (6,5), I can ask for the nearest object. The expected answer is Boxes (4,7), about 2.83 units away. If I ask for the distance to Shelf B, the answer is about 4.12 units.

I can also show an edge case by entering an object at coordinates that are already used. The program rejects the duplicate location. Another useful test is removing Shelf B and searching for (2,6) again; it should report that the object is no longer present.

12. What I Learned and Challenges

While working through the examples, I understood why the comparison changes between X and Y. I also saw that finding an exact point and finding the nearest point are different jobs. For nearest search, checking only one side of the tree is not always enough. These are the parts I would explain in my own words during the demonstration.

For the C++ program, I chose a simple remove method: collect the objects that should stay, clear the old tree, and insert those objects again. This is easier for me to follow than the specialized delete method in the LeetCode example [3], but it does more work. I also kept the robot position separate from the object locations.

13. What I Learned from the Three References

I used three online sources for different parts of the research. I did not copy their C++ programs into my project. Instead, I compared their explanations with the simpler 2D KD-Tree I built.

GeeksforGeeks [1]: This article helped me understand the basic structure. A KD-Tree is a binary tree of coordinate points, and the comparison axis changes with depth. Its insertion and search examples helped me understand why my program compares X at one level and Y at the next.

SSOJet [2]: This guide focuses on implementing a KD-Tree in C++. I learned that a node needs its point and links to its children, and that nearest-neighbor search may have to return to check another branch. It also describes building a tree using median points. My program is different because I insert objects in the order the user enters them, instead of building a balanced tree from a list.

LeetCode Discuss [3]: This community post shows C++ examples of insertion, exact search, and a more detailed delete operation. It helped me see that deleting a node is not as simple as removing one value from an array. My project uses a rebuild method instead, which is easier to demonstrate but less efficient.

Together, these sources helped me connect the theory to the code. The warehouse robot scenario, object names, menu options, and example coordinates are parts of my own project design, not examples claimed to come from those websites.

14. Conclusion

This assignment helped me understand how a KD-Tree can organize coordinate data and support useful searches. My C++ project demonstrates adding, finding, displaying, and removing objects, along with nearest-object and distance searches from a robot location. It is a simple simulation, but it shows a real idea used in spatial computing: organize points so nearby locations can be searched more efficiently.

References

[1] GeeksforGeeks. “KD Trees in C++.” Last updated January 20, 2026. https://www.geeksforgeeks.org/cpp/kd-trees-in-cpp/

[2] SSOJet. “Implement KD-Tree in C++.” Updated October 31, 2025. https://ssojet.com/dev-guides/implement-kd-tree-in-cpp

[3] fight.for.dream. “KD-tree Search & Insert & Delete Summary with C++ implementation.” LeetCode Discuss. https://leetcode.com/discuss/post/136595/kd-tree-search-insert-delete-summary-wit-5qzx/